Федеральное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники

Дисциплина: «Основы профессиональной деятельности»

Лабораторная работа №4
«Выполнение комплекса программ»
Вариант **3184**

Выполнил:
Крамарь Михаил Романович
P3113

Проверила:
Остапенко Ольга Денисовна

г. Санкт-Петербург
2026

Содержание

Текст задания 3
Ход работы 4
Оформление текста программы....4
Оформление текста подпрограммы....7
Назначение программы и реализуемая функция: 9
ОПИ и ОДЗ: 10
Таблица трассировки 12
Вывод....15

Текст задания

По выданному преподавателем варианту восстановить текст заданного варианта программы и подпрограммы (программного комплекса), определить предназначение и составить его описание, определить область представления и область допустимых значений исходных данных и результата, выполнить трассировку программного комплекса.

4EF: + 0200	4FD: 0800	50B: 0CA3	66C: 0B4E
4F0: EE1A	4FE: 4E0C	----------	66D: 00A7
4F1: AE17	4FF: EE0B	660: AC01	
4F2: 0700	500: AE07	661: F207	
4F3: 0C00	501: 0700	662: F006	
4F4: D660	502: 0C00	663: 7E08	
4F5: 0800	503: D660	664: F904	
4F6: 0700	504: 0800	665: 0500	
4F7: 6E13	505: 4E05	666: 4C01	
4F8: EE12	506: EE04	667: 4E05	
4F9: AE10	507: 0100	668: CE01	
4FA: 0740	508: ZZZZ	669: AE02	
4FB: 0C00	509: YYYY	66A: EC01	
4FC: D660	50A: XXXX	66B: 0A00	

Ход работы
Оформление текста программы

Адрес	*Код команды*	*Мнемоника*	*Комментарии*
4EF	**0200**	**CLA**	**Очистка аккумулятора: 0 -> AC (Очистить аккумулятор)**
4F0	**EE1A**	**ST IP+26**	**Прямое относительное сохранение: AC -> 50B (Происходит сложение закодированного в команде смещения (+26) со счётчиком команд; IP указывает на 4BD)**
4F1	**AE17**	**LD+21**	**Прямая относительная загрузка: 509 -> AC (работаем с Y) (Адрес операнда получается сложением закодированного в команде смещения (+21) со счётчиком команд; IP указывает на 4F2)**
4F2	**0700**	**INC**	**Инкрементирование аккумулятора: AC + 1 -> AC**
4F3	**0C00**	**PUSH**	**Положить на стек: AC -> -(SP) (Указатель стека изначально сброшен в 0, после указывает на 7FF, записывает туда значение AC)**
4F4	**D660**	**CALL 660**	**Вызов подпрограммы: SP - 1 -> SP, IP -> (SP), M -> IP (Указатель стека теперь указывает на 7FE, вместо 7FF; Запись IP = 4F5 в ячейку 7FE стека; переход к подпрограмме по адресу 660)**
4F5	**0800**	**POP**	**Снятие со стека: (SP)+ -> AC (По адресу, содержащемуся в SP, читает значение из памяти, а потом увеличивает значение SP = SP + 1; AC получает результат f(Y+1))**
4F6	**0700**	**INC**	**Инкрементирование аккумулятора: AC + 1 -> AC**

4F7	**6E13**	**SUB IP+19**	**Прямое относительное вычитание из аккумулятора: AC - 50B -> AC (Адрес операнда получается сложением закодированного в команде смещения (+19) со счётчиком команд; IP указывает на 4F8)**
4F8	**EE12**	**ST IP+18**	**Прямое относительное сохранение: AC -> 50B (Адрес операнда получается сложением закодированного в команде смещения (+18) со счётчиком команд; IP указывает на 4F9)**
4F9	**AE10**	**LD IP+16**	**Прямая относительная загрузка: 50A -> AC (работаем с X) (Адрес операнда получается сложением закодированного в команде смещения (+16) со счётчиком команд; IP указывает на 4FA)**
4FA	**0740**	**DEC**	**Декрементирование аккумулятора: AC - 1 -> AC**
4FB	**0C00**	**PUSH**	**Положить на стек: AC -> -(SP) (Указатель стека изначально сброшен в 0, после указывает на 7FF, записывает туда значение AC)**
4FC	**D660**	**CALL 660**	**Вызов подпрограммы: SP - 1 -> SP, IP -> (SP), M -> IP (Указатель стека теперь указывает на 7FE, вместо 7FF; Запись IP = 4FD в ячейку 7FE стека; переход к подпрограмме по адресу 660)**
4FD	**0800**	**POP**	**Снятие со стека: (SP)+ -> AC (По адресу, содержащемуся в SP, читает значение из памяти, а потом увеличивает значение SP = SP + 1; AC получает результат f(X-1))**

4FE	**4E0C**	**ADD IP+12**	**Прямое относительное сложение: 50B + AC -> AC (Адрес операнда получается сложением закодированного в команде смещения (+12) со счётчиком команд; IP указывает на 4FF)**
4FF	**EE0B**	**ST IP+11**	**Прямое относительное сохранение: AC -> 50B (Адрес операнда получается сложением закодированного в команде смещения (+11) со счётчиком команд; IP указывает на 50B)**
500	**AE07**	**LD IP+7**	**Прямая относительная загрузка: 508 -> AC (работаем с Z) (Адрес операнда получается сложением закодированного в команде смещения (+7) со счётчиком команд; IP указывает на 501)**
501	**0700**	**INC**	**Инкрементирование аккумулятора: AC + 1 -> AC**
502	**0C00**	**PUSH**	**Положить на стек: AC -> -(SP) (Указатель стека изначально сброшен в 0, после указывает на 7FF, записывает туда значение AC)**
503	**D660**	**CALL 660**	**Вызов подпрограммы: SP - 1 -> SP, IP -> (SP), M -> IP (Указатель стека теперь указывает на 7FE, вместо 7FF; Запись IP = 504 в ячейку 7FE стека; переход к подпрограмме по адресу 660)**
504	**0800**	**POP**	**Снятие со стека: (SP)+ -> AC (По адресу, содержащемуся в SP, читает значение из памяти, а потом увеличивает значение SP = SP + 1; AC получает результат f(Z+1))**

505	**4E05**	**ADD IP+5**	**Прямое относительное сложение: 50B + AC -> AC (Адрес операнда получается сложением закодированного в команде смещения (+5) со счётчиком команд; IP указывает на 506)**
506	**EE04**	**ST IP+4**	**Прямое относительное сохранение: AC -> 50B (Адрес операнда получается сложением закодированного в команде смещения (+4) со счётчиком команд; IP указывает на 507)**
507	**0100**	**HLT**	**Останов. Переход в пультовый режим**
508	ZZZZ	-	Значение Z
509	YYYY	-	Значение Y
50A	XXXX	-	Значение X
50B	0CA3	-	Результат R

Оформление текста подпрограммы

Адрес	*Код команды*	*Мнемоника*	*Комментарии*
660	**AC01**	**LD SP+1**	**Загрузка со смещением относительно SP: 7FF -> AC (Адрес операнда получается сложением закодированного в команде смещения (+1) с указателем стека; SP указывает на 7FE; IP указывает на 661)**
661	**F207**	**BMI (BNS) IP+7**	**Переход, если минус (N==1): (Перед этим IP указывает на 662) IP = IP + 7 (К адресу 669)**

662	**F006**	**BEQ (BZS) IP+6**	**Переход, если равенство (Z==1): (Перед этим IP указывает на 663) IP = IP + 6 (К адресу 669)**
663	**7E08**	**CMP IP+8**	**Прямая относительная установка флагов по результату: AC - 66C (Адрес операнда получается сложением закодированного в команде смещения (+8) со счётчиком команд; IP указывает на 664)**
664	**F904**	**BGE IP+4**	**Переход, если больше или равно (N xor V == 0 / N == V): (Перед этим IP указывает на 665) IP = IP + 4 (К адресу 669)**
665	**0500**	**ASL**	**Арифметический сдвиг влево: AC сдвигается влево; AC15 -> C, 0 -> AC0 (Эквивалентно умножению на 2)**
666	**4C01**	**ADD SP+1**	**Сложение со смещением относительно SP: 7FF + AC -> AC (Адрес операнда получается сложением закодированного в команде смещения (+1) с указателем стека; SP указывает на 7FE; IP указывает на 667)**
667	**4E05**	**ADD IP+5**	**Прямое относительное сложение: 66D + AC -> AC (Адрес операнда получается сложением закодированного в команде смещения (+5) со счётчиком команд; IP указывает на 668)**
668	**CE01**	**JUMP IP+1**	**Прямой относительный прыжок: 66A -> IP (Адрес получается сложением закодированного в команде смещения (+1) со счётчиком команд; IP указывает на 669)**

669	**AE02**	**LD M**	**Прямая относительная загрузка: 66C -> AC (Адрес операнда получается сложением закодированного в команде смещения (+2) со счётчиком команд; IP указывает на 66A)**
66A	**EC01**	**ST SP+1**	**Сохранение со смещением относительно стека: AC -> 7FF (Адрес операнда получается сложением закодированного в команде смещения (+1) с указателем стека; SP указывает на 7FE; IP указывает на 66B)**
66B	**0A00**	**RET**	**Выход из подпрограммы: (SP) + -> IP (Переход к адресу, сохранённому в текущей ячейке стека, SP = SP + 1)**
66C	0B4E	-	A
66D	00A7	-	B

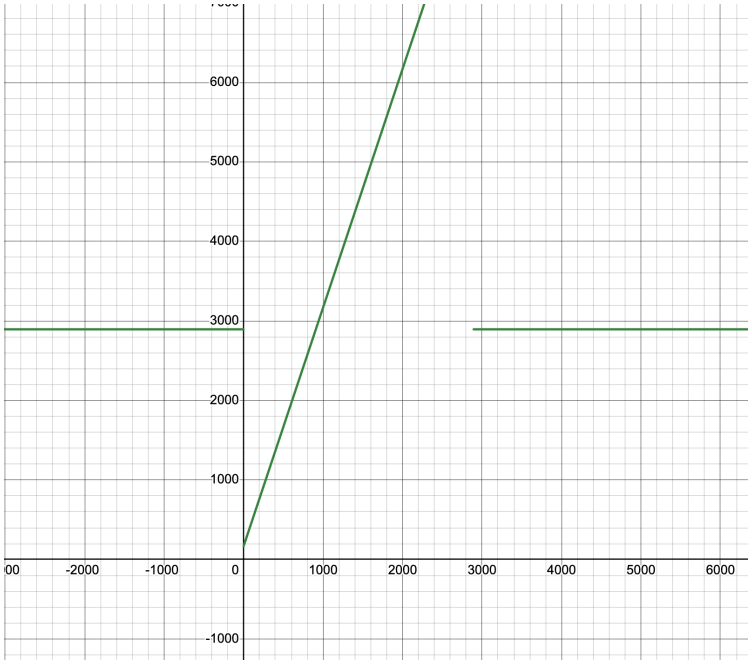
Назначение программы и реализуемая функция:

- Нахождение значения суммы функции *f* от трёх переменных + единица:
  $R = f(X - 1) + f(Y + 1) + f(Z + 1) + 1$
- R - результат подсчёта;
- X, Y, Z - переменные (исходные числа);
- A, B - заданные константы в функции *f;*
  $A = 0B4E_{16} = 2894_{10}$
  $B = 00A7_{16} = 167_{10}$

Формула:

$$f(x) = \begin{cases} 2894, & при\ x \le 0\ или\ x \ge 2894 \\ 3x + 167, & при\ 0 < x < 2894 \end{cases}$$

График f(x) (функции, которую реализует подпрограмма):



ОПИ и ОДЗ:

- ***Область представления***:
- X, Y, Z, A, B, R - 16-ти разрядные знаковые числа;
- ***Область допустимых значений:***
- Чтобы получить максимально правильную ОДЗ, сначала проанализируем функцию f(x), которую реализует подпрограмма:

  При x, лежащем в промежутке [-2^{15}; 0] или [2894; 2^{15} - 1], значение функции будет константой A, то есть функция возвращает значение 2894. Получается, что при использовании аргумента из приведённых промежутков, переполнения не возникает.

  При x, принадлежащему оставшемуся промежутку (0; 2894), функция должна вернуть значение 3x + 167, которое должно входить в диапазон [-2^{15}; 2^{15} - 1]. На заданном промежутке функция является монотонно возрастающей (так как это прямая), поэтому посмотрим на её максимальное и минимальное значение:
  $f_{min} = f(1) = 170$
  $f_{max} = f(2893) = 8846$

  То есть результат функции лежит на отрезке [170; 8846] при заданном промежутке для аргумента, этот отрезок входит в [-2^{15}; 2^{15} - 1].

- Итак, рассмотрим R:
  $R = f(X - 1) + f(Y + 1) + f(Z + 1) + 1$
  Минимально мы можем получить R = 170 + 170 + 170 + 1 = 511
  Максимально мы можем получить R = 8846 + 8846 + 8846 + 1 = 26539

Переполнения в обоих случаях нет!

- ***Само ОДЗ:***

$$\begin{cases} X \in [-2^{15}+1; 2^{15}-1] \\ Y, Z \in [-2^{15}; 2^{15}-2] \\ R \in [511; 26539] \text{ (учитывая заданные A и B)} \end{cases}$$

- **Расположение данных и программы в памяти ЭВМ**

Основная программа:

- Команды: ***4EF - 507;***
- Исходные данные: Z - ***508***, Y - ***509***, X - ***50A***;
- Итоговый результат: R - ***50B***;

Подпрограмма:

- Команды: ***660 - 66B***;
- Исходные данные: A - ***66C***, B - ***66D***;

- **Адреса первой и последней исполняемых команд**

Основная программа:

- Первая команда: ***4EF***;
- Последняя команда: ***507***;

Подпрограмма:

- Первая команда: ***660***;
- Последняя команда: ***66B***;

Таблица трассировки

Значения *(взял так, чтобы попались сразу три случая просто):*

$X = A100_{16}$; $Y = 24B8_{16}$; $Z = 0ABB_{16}$;

Выполняемая команда		Содержимое регистров процессора после выполнения команды								Ячейка, содержимое которой изменилось после выполнения команды	
Адрес	Код	IP	CR	AR	DR	SP	BR	AC	NZVC	Адрес	Новый код
4EF	+0200	4F0	0200	4EF	0200	000	04EF	0000	0100	-	-
4F0	EE1A	4F1	EE1A	50B	0000	000	001A	0000	0100	50B	0000
4F1	AE17	4F2	AE17	509	24B8	000	0017	24B8	0000	-	-
4F2	0700	4F3	0700	4F2	0700	000	04F2	24B9	0000	-	-
4F3	0C00	4F3	0C00	7FF	24B9	7FF	04F3	24B9	0000	7FF	24B9
4F4	D660	660	D660	7FE	04F5	7FE	D660	24B9	0000	7FE	04F5
660	AC01	661	AC01	7FF	24B9	7FE	0001	24B9	0000	-	-
661	F207	662	F207	661	F207	7FE	0661	24B9	0000	-	-
662	F006	663	F006	662	F006	7FE	0662	24B9	0000	-	-
663	7E08	664	7E08	66C	0B4E	7FE	0008	24B9	0001	-	-
664	F904	665	F904	669	F904	7FE	0004	24B9	0001	-	-
669	AE02	66A	AE02	66C	0B4E	7FE	0002	0B4E	0001	-	-
66A	EC01	66B	EC01	66C	0B4E	7FE	0001	0B4E	0001	7FF	0B4E
66B	0A00	4F5	0A00	7FE	04F5	7FF	066B	0B4E	0001	-	-
66C	0B4E	-	-	-	-	-	-	-	-	-	-
66D	00A7	-	-	-	-	-	-	-	-	-	-
4F5	0800	4F6	0800	7FF	0B4E	000	04F5	0B4E	0001	-	-
4F6	0700	4F7	0700	4F6	0700	000	04F6	0B4F	0000	-	-
4F7	6E13	4F8	6E13	50B	0000	000	0013	0B4F	0001	-	-
4F8	EE12	4F9	EE12	50B	0B4F	000	0012	0B4F	0001	50B	0B4F
4F9	AE10	4FA	AE10	50A	A100	000	0010	A100	1001	-	-
4FA	0740	4FB	0740	4FA	0740	000	04FA	A0FF	1001	-	-
4FB	0C00	4FC	0C00	7FF	A0FF	7FF	04FB	A0FF	1001	7FF	A0FF
4FC	D660	660	D660	7FE	04FD	7FE	D660	A0FF	1001	7FE	04FD
660	AC01	661	AC01	7FF	A0FF	7FE	0001	A0FF	1001	-	-

661	F207	662	F207	669	F207	7FE	0007	A0FF	1001	-	-
669	AE02	66A	AE02	66C	0B4E	7FE	0002	0B4E	0001	-	-
66A	EC01	66B	EC01	7FF	0B4E	7FE	0001	0B4E	0001	7FF	0B4E
66B	0A00	4FD	0A00	7FE	04FD	7FF	066B	0B4E	0001	-	-
66C	0B4E	-	-	-	-	-	-	-	-	-	-
66D	00A7	-	-	-	-	-	-	-	-	-	-
4FD	0800	4FE	0800	7FF	0B4E	000	04FD	0B4E	0001	-	-
4FE	4E0C	4FF	4E0C	50B	0B4F	000	000C	169D	0000	-	-
4FF	EE0B	500	EE0B	50B	169D	000	000B	169D	0000	50B	169D
500	AE07	501	AE07	508	0ABB	000	0007	0ABB	0000	-	-
501	0700	502	0700	501	0700	000	0501	0ABC	0000	-	-
502	0C00	503	0C00	7FF	0ABC	7FF	0502	0ABC	0000	7FF	0ABC
503	D660	660	D660	7FE	0504	7FE	D660	0ABC	0000	7FE	0504
660	AC01	661	AC01	7FF	0ABC	7FE	0001	0ABC	0000	-	-
661	F207	662	F207	661	F207	7FE	0661	0ABC	0000	-	-
662	F006	663	F006	662	F006	7FE	0662	0ABC	0000	-	-
663	7E08	664	7E08	66C	0B4E	7FE	0008	0ABC	1000	-	-
664	F904	665	F904	664	F904	7FE	0664	0ABC	1000	-	-
665	0500	666	0500	665	0ABC	7FE	0665	1578	0000	-	-
666	4C01	667	4C01	7FF	0ABC	7FE	0001	2034	0000	-	-
667	4E05	668	4E05	66D	00A7	7FE	0005	20DB	0000	-	-
668	CE01	66A	CE01	668	066A	7FE	0001	20DB	0000	-	-
66A	EC01	66B	EC01	7FF	20DB	7FE	0001	20DB	0000	7FF	20DB
66B	0A00	504	0A00	7FE	0504	7FF	066B	20DB	0000	-	-
66C	0B4E	-	-	-	-	-	-	-	-	-	-
66D	00A7	-	-	-	-	-	-	-	-	-	-
504	0800	505	0800	7FF	20DB	000	0504	20DB	0000	-	-
505	4E05	506	4E05	50B	169D	000	0005	3778	0000	-	-
506	EE04	507	EE04	50B	3778	000	0004	3778	0000	50B	3778
507	0100	508	0100	507	0100	000	0507	3778	0000	-	-
508	0ABB	-	-	-	-	-	-	-	-	-	-
509	24B8	-	-	-	-	-	-	-	-	-	-
50A	A100	-	-	-	-	-	-	-	-	-	-
50B	0CA3	-	-	-	-	-	-	-	-	-	-

- ***Практический результат:***
  При $X = A100_{16} = -24320_{10}$, $Y = 24B8_{16} = 9400_{10}$, $Z = 0ABB_{16} = 2747_{10}$:
  Мы получаем $R = 3778_{16} = \mathbf{14200}_{10}$

- ***Теоретический результат:***
  Вычисление теоретического результата с нашими X, Y, Z:
  $R = f(-24320 - 1) + f(9400 + 1) + f(2747 + 1) + 1 = 2894 + 2894 + 3 * 2748 + 167 + 1 = \mathbf{14200}_{10} = 3778_{16}$

Теоретический результат совпал с результатом, полученным на практике!!! Результат входит в теоретическое ОДЗ.

Вывод

В ходе выполнения лабораторной работы №4 я научился работать со стеком, узнал про подпрограммы, ещё раз закрепил свои знания о видах адресации в БЭВМ на практике. Изучил цикл выполнения новых для меня команд - CALL и RET, узнал как работать со стеком при помощи команд POP и PUSH.